Scope-based authorization policies

In section 25.2 I described the role of scopes in the authentication process. When you obtain a bearer token from an identity provider—whether you're using OpenID Connect or OAuth 2.0—you define the scopes that you wish to retrieve. The user can then choose to grant or deny some or all of those requested scopes. Additionally, the identity provider might allow certain client applications access only to specific scopes. The final access token you receive from the identity provider, which is sent to the API, may have some or none of the requested scopes.

It's up to the API itself to decide what each scope means and how it should be used to enforce authorization policies. Scopes have no inherent functionality on their own, much like claims, but you can build functionality on top. For example, you can create authorization polices that require a token has the scope `"recipe.edit"` using

```
builder.Services.AddAuthorizationBuilder()
    .AddPolicy("RecipeEditScope", policy =>
        policy.RequireClaim("scope", " recipe.edit "));
```

This policy could then be applied to any endpoints that edit a recipe.

Another common pattern is to require a specific scope for you to be authorized to make any requests to a given ASP.NET Core app, such as a `"receipeApi"` scope. This approach can often replace audience validation in bearer token authorization and may be more flexible, as it doesn't require your identity provider to know the domain at which your API app will be hosted.

Alternatively, you can use scopes to partition your APIs into groups that can only be accessed by certain types of clients. For example, you might have one set of APIs that can be accessed only by internal machine-to-machine clients, another set that can be accessed only by admin users, and another set that can be accessed only by nonadmin users.

Duende has many practical examples of approaches to authorization and authentication using OpenID Connect at http://mng.bz/o1Jp. The examples are geared to IdentityServer users but show many best practices and patterns you can use with identity provider services as well.

That brings us to the end of this chapter on authentication and authorization. We're not completely done with security, though; in chapter 27 we look at